SOFTWARE ENGINEERING

UNIT – I

Introduction: Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
Software Life Cycle Models: Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

WHAT IS SOFTWARE ENGINEERING?

The term **software engineering** is the product of two words, **software**, and **engineering**.

The **SOFTWARE** is a collection of integrated programs.
Software subsists of carefully-organized instructions and code written by developers on any of various particular computer languages.
ENGINEERING is the application of scientific and practical knowledge to invent, design, build, maintain, and improve frameworks, processes, etc.

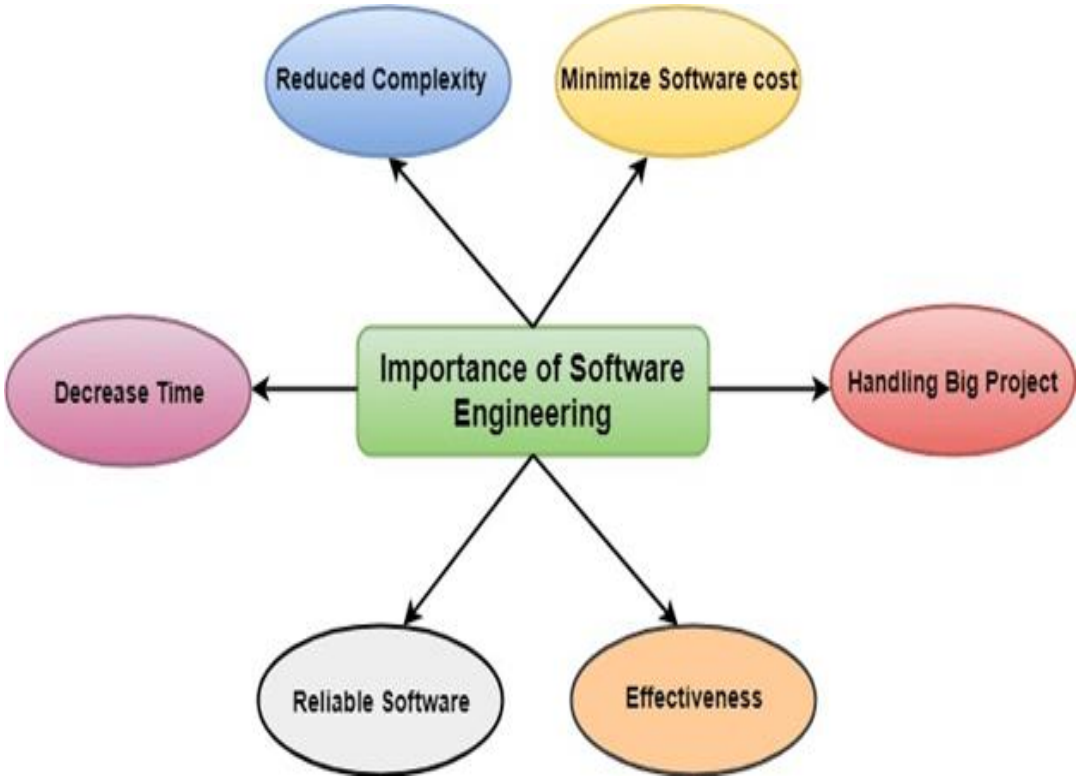


SOFTWARE ENGINEERING DEFINITION:

Software engineering the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software

A popular definition of software engineering is: "A systematic collection of good program development practices and techniques". Good program development techniques have resulted from research innovations as well as from the lessons learnt by programmers through years of programming experiences. An alternative definition of software engineering is: "An engineering approach to develop software". Let us now try to figure out what exactly is meant by an engineering approach to develop software. We explain this using an analogy.

IMPORTANCE OF SOFTWARE ENGINEERING:



The importance of Software engineering is as follows:

1. **Reduces complexity:** Large software projects are inherently complex. Software engineering breaks down these projects into smaller, manageable components, simplifying development.
2. **Minimizes software cost:** By careful planning and eliminating unnecessary elements, software engineering reduces the overall cost of software production compared to ad-hoc development.
3. **Decreases time:** Proper planning and methodology reduce wasted time associated with trial-and-error coding, ensuring more efficient development cycles.

4. **Handles big projects:** Large projects require extensive planning, management, testing, and maintenance over extended periods. Software engineering provides the necessary framework for effectively managing these complex undertakings.
5. **Reliable software:** Software engineering emphasizes testing and maintenance, ensuring delivered software is dependable, secure, and functions as expected.
6. **Effectiveness:** By adhering to software standards, software engineering helps create more effective and high-quality products.

EVOLUTION:

Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities. The software evolution process includes fundamental activities of change analysis, release planning, system implementation, and releasing a system to customers.

1. The cost and impact of these changes are accessed to see how much the system is affected by the change and how much it might cost to implement the change.
2. If the proposed changes are accepted, a new release of the software system is planned.
3. During release planning, all the proposed changes are considered.
4. A design is then made on which changes to implement in the next version of the system.
5. The process of change implementation is an iteration of the development process where the revisions to the system are designed, implemented, and tested.

1945-1965: Origin

- **Early Computing and Theoretical Foundations**
  - The period saw the birth of modern computing, with the development of the first electronic computers like ENIAC (1945), and theoretical advancements in computing (e.g., Turing machines).
  - The first stored-program computers emerged, and key concepts like binary arithmetic, algorithms, and computational logic started to form the foundation for later developments.

1965-1985: Crisis

- **Challenges in Scaling and Computing Limits**
  - The "Crisis" might refer to the difficulty in scaling up hardware to meet the growing demand for computational power and memory.
  - During this period, transistor technology advanced, but the complexity of systems and rising costs became problematic for both research and commercial use because 2%success rate in software products due to time line problem.

- Time line means in the period of time duration if we not completed the project properly than that project goes to scrap. Ex OS/360(It is a millionaire operating system project)

1985-2000: Internet

- **Global Connectivity and the Digital Revolution**
  - The rise of the Internet revolutionized computing by introducing global communication, online services, and e-commerce.
  - The development of the World Wide Web (1991) made the Internet accessible to the masses, reshaping business, culture, and communication, making computers more user-friendly.

2000-2010: Lightweight

- **Portable Devices and Efficiency**
  - This era marked the proliferation of mobile computing and lightweight devices. Laptops, smart phones, and note books became more popular, emphasizing portability and battery efficiency.
  - The rise of cloud computing began to take hold, allowing for lightweight computing experiences by offloading processing to remote servers, which also spurred innovation in online storage and services.

2010-Present: Tile AI, ML, DL

- **Artificial Intelligence, Machine Learning, and Deep Learning**
  - This period marks the explosion of Artificial Intelligence (AI), particularly in the form of machine learning (ML) and deep learning (DL).
  - Machine learning algorithms began outperforming traditional software solutions in areas like speech recognition, image processing, and natural language understanding.
  - Cloud platforms like Google Cloud, AWS, and Microsoft Azure made AI and ML tools more accessible to businesses and researchers, democratizing these technologies.

➢ **Evolution of an Art into an Engineering Discipline**

Software engineering has transitioned from an "art" to a "craft" and finally to an "engineering discipline" over the past sixty years. Early programming was characterized by ad hoc styles like "exploratory," "build and fix," and "code and fix." Exploratory programming is informal, relying on individual intuition and experience, often resulting in poor quality, unmaintainable, and expensive code. The "build and fix" style, common in early computing, relied on some programmers' "esoteric knowledge" to produce good programs, while others struggled. Modern software development, however,

emphasizes applying well-understood principles rather than relying on such individual, intuitive approaches.

- **Evolution Pattern for Engineering Disciplines**

Initially, programming knowledge was often secret and passed down within small groups. This transitioned to a craft where knowledge was shared through apprenticeships. Eventually, systematic organization, documentation, and scientific principles led to the emergence of software engineering. While some criticize software engineering methodologies for lacking scientific basis and being subjective, its practices are widely accepted for facilitating the development of high-quality software efficiently, especially for large projects. Smaller programs, however, can still be successfully developed using exploratory styles.

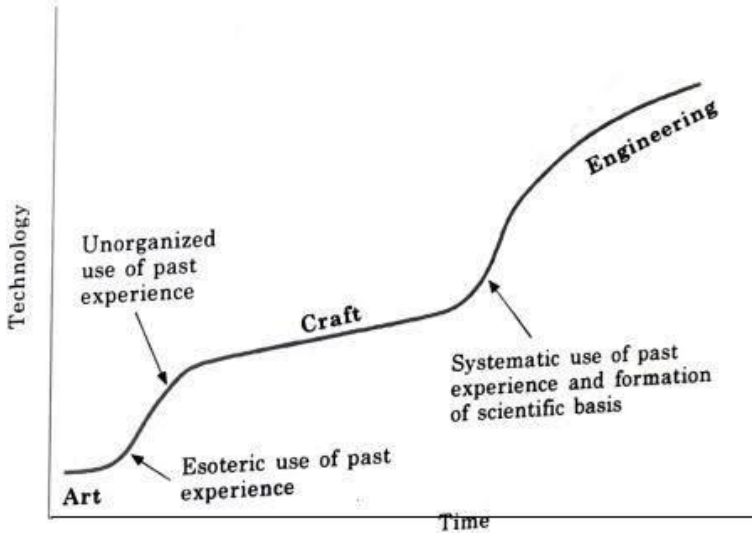


FIGURE: Evolution of technology with time.

- **A Solution to the Software Crisis**

The increasing cost of software compared to hardware is a major symptom of the current "software crisis." Organizations are spending a growing portion of their budgets on software, a trend that could lead to a future where hardware is essentially given away with software purchases. Several factors contribute to this crisis: rapidly increasing problem size, inadequate training in software engineering, a growing skills shortage, and low productivity improvements. Solutions are believed to lie in wider adoption of software engineering practices and further advancements within the discipline itself.

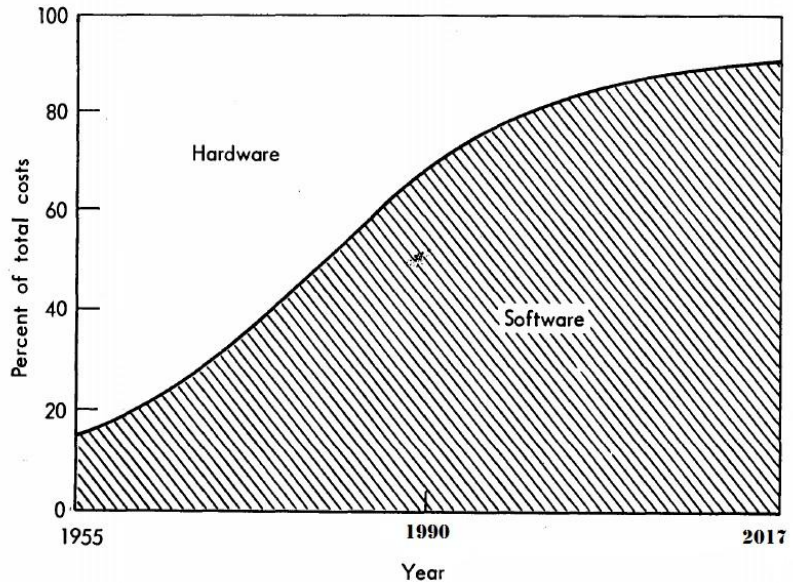


FIGURE: Relative changes of hardware and software costs over time.

SOFTWARE DEVELOPMENT PROJECTS:

❖ **Programs** Vs. **Software Products**

Programs (Toy Software)

- **Developed by:** Individuals (students, hobbyists)
- **Purpose:** Learning, personal use
- **Characteristics:**
  - Small size, limited functionality programs
  - Poor UI, no documentation
  - Low maintainability, efficiency, reliability
  - Primarily used by the creator

Software Products

- **Developed by:** Teams of professionals
- **Purpose:** Commercial use, large user base
- **Characteristics:**
  - Large size, complex systems
  - Good UI, extensive documentation
  - High maintainability, efficiency, reliability
  - Systematically designed, tested

Key Differences

- **Scope:** Programs are small; products are large.
- **Development:** Programs are individual; products require teams and methodologies.
- **Users:** Programs have limited users; products have large user bases.

- **Documentation:** Programs lack documentation; products have extensive documentation.
- **Quality:** Programs may have lower quality; products prioritize quality and standards.

❖ **Types of Software Development Projects:**

A software development company is typically structured into a large number of teams that handle various types of software development projects. These software deve lo pme nt projects concern the development of 2 types.

1. Software product
2. Software service.

Software products

We all know of a variety of software such as Microsoft's Windows and the Office suite, Oracle DBMS, software accompanying a camcorder or a laser printer, etc. These software are available off-the-shelf for purchase and are used by a diverse range of customers. These are called generic software products since many users essentially use the same software. These can be purchased off-the-shelf by the customers. When a software development company wishes to develop a generic product, it first determines the features or functionalities that would be useful to a large cross section of users. Based on these, the development team draws up the product specification on its own. Of course, it may base its design discretion on feedbacks collected from a large number of users. Typically, each software product is targetted to some market segment(set of users).Many companies find it advantageous to develop product lines that target slightly different market segments based on variations of essentially the same software. For example, Microsoft targets desktops and laptops through its Windows 8 operating system, while it targets high-end mobile handsets through its Windows mobile operating system, and targets servers through its Windows server operating system.

Software services

Software Services encompass two primary categories:

1. Customized Software Development:

- **Tailored Solutions:** These are developed to meet the specific needs of a particular client or a small group of clients.
- **Rapid Development:** Requires quick turnaround times (typically within a few months).
- **Cost-Effectiveness:** Development costs need to be kept low.
- **Reusing Existing Code:** Companies often adapt existing software to meet the client's requirements, minimizing development from scratch.
- **Example:** An academic institution needs software for student registration, grading, and fee collection. The development company would likely adapt an existing academic software solution to fit the institution's specific needs.

2. Outsourced Software Development:

- **Part of a Larger Project:** Companies outsource specific components of their larger software development projects.
- **Reasons for Outsourcing:**
  - Lack of internal expertise in certain areas.
  - Cost-effectiveness of development by another company.
- **Small Projects:** Outsourced projects are usually smaller in scope and have shorter deadlines (weeks or months).

Key Takeaways:

- Both types of software services focus on meeting specific client needs.
- Customized software involves adapting existing solutions, while outsourced development focuses on specific components within a larger project.
- Time and cost constraints are crucial factors in both types of services.

EXPLORATORY STYLE OF SOFTWARE DEVELOPMENT:

We have already discussed that the exploratory program development style refers to an informal development style where the programmer makes use of his own intuition to develop a program rather than making use of the systematic body of knowledge categorized under the software engineering discipline. The exploratory development style gives complete freedom to the programmer to choose the activities using which to develop software. Though the exploratory style imposes no rules a typical development starts after an initial briefing from the customer. Based on this briefing, the developers start coding to develop a working program. The software is tested and the bugs found are fixed. This cycle of testing and bug fixing continues till the software works satisfactorily for the customer. A schematic of this work sequence in a build and fix style has been shown graphically in below figure. Observe that coding starts after an initial customer briefing about what is required. After the program development is complete, a test and fix cycle continues till the program becomes acceptable to the customer.

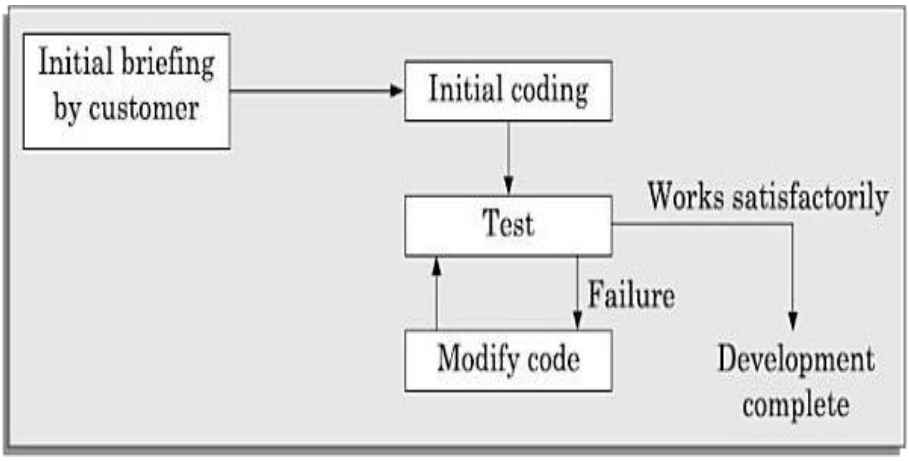


Figure: Exploratory program development.

Summary of the short comings of the exploratory style of software development:

We briefly summarize the important shortcomings of using the exploratory development style to develop a professional software.

- The fore most difficulty is the exponential growth of development time and effort with problem size and large-sized software becomes almost impossible using this style of development.
- The exploratory style usually results in unmaintainable code. The reason for this is that any code developed without proper design would result in highly unstructured and poor quality code.
- It becomes very difficult to use the exploratory style in a team development environment. In the exploratory style, the development work is undertaken without any proper design and documentation. Therefore it becomes very difficult to meaningfully partition the work among a set of developers who can work concurrently. On the other hand, team development is indispensable for developing modern software—most software mandate huge development efforts, necessitating team effort for developing these. Besides poor quality code, lack of proper documentation makes any later maintenance of the code very difficult.

EMERGENCE OF SOFTWARE ENGINEERING:

We have already pointed out that software engineering techniques have evolved over many years in the past. This evolution is the result of a series of innovations and accumulation of experience about writing good quality programs. Since these innovations and programming experiences are too numerous, let us briefly examine only a few of these innovations and programming experiences which have contributed to the development of the software engineering discipline

Early Computer Programming:

Early commercial computers were very slow and too elementary as compared to today's standards. Even simple processing tasks took considerable computation time on those computers. No wonder that programs at that time were very small in size and lacked sophistication. Those programs were usually written in assembly languages. Program lengths were typically limited to about a few hundreds of lines of monolithic assembly code. Every programmer developed his own individualistic style of writing programs according to his intuition and used this style ad hoc while writing different programs. In simple words, programmers wrote programs without formulating any proper solution strategy, plan, or design a jump to the terminal and start coding immediately on hearing out the problem. They then went on fixing any problems that they observed until they had a program that worked reasonably well. We have already designated this style of programming as the build and fix (or the exploratory programming) style.

High-level Language Programming:

Computers became faster with the introduction of the semiconductor technology in the early 1960s.Faster semi conductor transistors replaced the prevalent vacuum tube-based circuits in a computer. With the availability of more powerful computers, it became possible to solve larger and more complex problems.

At this time, high-level languages such as FORTRAN, ALGOL, and COBOL were introduced. This considerably reduced the effort required to develop software and helped programmers to write larger programs (why?). Writing each high-level programming construct in effect enables the programmer to write several machine instructions. Also, the machine details (registers, flags, etc.) are abstracted from the programmer. However, programmers were still using the exploratory style of software development. Typical programs were limited to sizes of around a few thousands of lines of source code.

Control Flow-based Design:

As the size and complexity of programs kept on increasing, the exploratory programming style proved to be insufficient. Programmers found it increasingly difficult not only to write cost-effective and correct programs, but also to understand and maintain programs written by others. To cope up with this problem, experienced programmers advised other programmers to pay particular attention to the design of a program's control flow structure.

A program's control flow structure indicates the sequence in which the program's instructions are executed.

In order to help develop programs having good control flow structures, the flowcharting technique was developed. Even today, the flowcharting technique is being used to represent and design algorithms; though the popularity of low charting represent and design programs has want to a great extent due to the emergence of more advanced techniques.

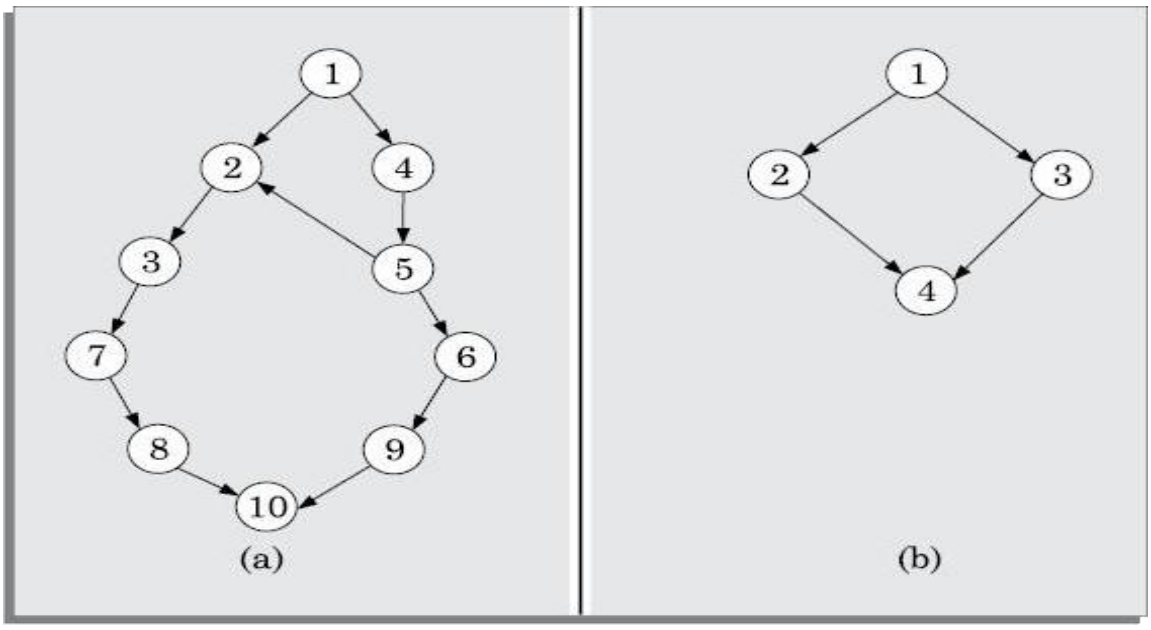


Figure 1.10: Control flow graphs of the programs.

Let us now try to understand why a program having good control flow structure would be easier to develop and understand. In other words, let us understand why a program with a complex flow chart representation is difficult to understand? The main reason behind this situation is that normally one understands a program

by mentally tracing its execution sequence (i.e. statement sequences) to understand how the output is produced from the input values. That is, we can start from a statement producing an output, and trace back the statements in the program and understand how they produce the output by transforming the input data. Alternatively, we may start with the input data and check by running through the program how each statement processes (transforms) the input data until the output is produced. For example, for the program of Fig 1.10(a) you would have to understand the execution of the program along the paths 1-2-3-7-8-10, 1-4-5-6-9-10, and 1- 4-5-2-3-7-8-10. A program having a messy control flow(i.e. flow chart) structure, would have a large number of execution paths (see Figure 1.11).

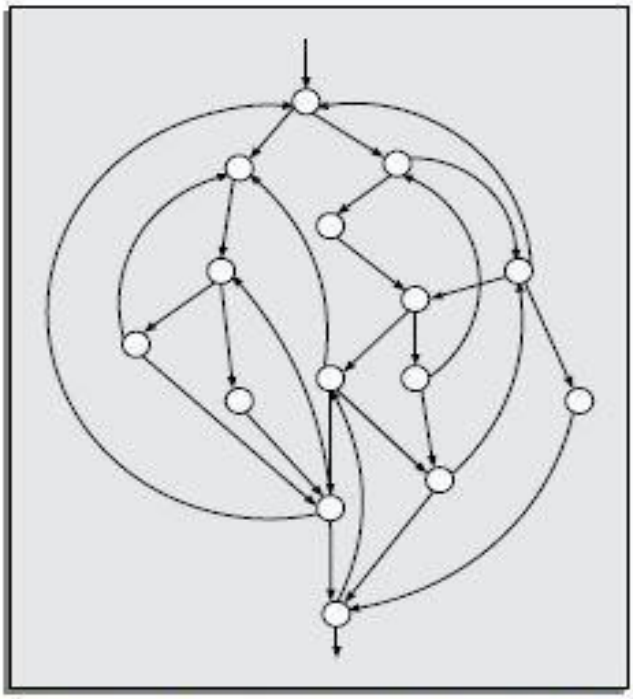


FIGURE 1.11 CFG of a program having too many GO TO statements.

Consequently, it would become extremely difficult to determine all the execution paths, and tracing the execution sequence along all the paths trying to understand them can be nightmarish. It is therefore evident that a program having a messy flow chart representation would indeed be difficult to understand and debug.

NOTABLE CHANGES IN SOFTWARE DEVELOPMENT PRACTICES:

The provided text outlines key shifts in software development practices, moving away from exploratory, ad-hoc approaches towards more structured and disciplined methodologies. Here's a summary of the notable changes:

1. From Error Correction to Error Prevention:

- **Exploratory:** Primarily focused on fixing bugs after they occur ("build and fix").

- **Modern:** Emphasizes proactive error prevention through careful planning, design, and testing. This is recognized as more cost-effective than fixing issues later in the development cycle.

2. Beyond Coding:

- **Exploratory:** Viewed coding as the primary development activity.
- **Modern:** Recognizes coding as one part of a larger process, with significant emphasis on design, testing, and other crucial activities.

3. Importance of Requirements Specification:

- **Exploratory:** Less emphasis on clearly defining requirements upfront.
- **Modern:** Prioritizes thorough and accurate requirements gathering to minimize costly rework later in the development cycle.

4. Regular Reviews and Inspections:

- **Exploratory:** Limited emphasis on formal reviews and inspections.
- **Modern:** Incorporates regular reviews throughout the development process to identify and address issues early on ("phase containment of errors").

5. Focus on Documentation:

- **Exploratory:** Minimal documentation of design, testing, and other activities.
- **Modern:** Emphasizes the creation of high-quality, consistent documentation throughout the development process. This improves maintainability, fault diagnosis, and overall project understanding.

6. Project Planning and Resource Management:

- **Exploratory:** Less emphasis on detailed project planning and resource allocation.
- **Modern:** Prioritizes careful planning to ensure timely execution of development activities and efficient resource utilization.

COMPUTER SYSTEMS ENGINEERING:

In all the discussions so far, we assumed that the software being developed would run on some general-purpose hardware platform such as a desktop computer or a server. But, in several situations it may be necessary to develop special hardware on which the software would run. Examples of such systems are numerous, and include a robot, a factory automation system, and a cell phone. In a cell phone, there is a special processor and other specialized devices such as a speaker and a microphone. It can run only the programs written specifically for it. Development of such systems entails development of both software and specific hardware that would run the software. Computer systems engineering addresses development of such systems requiring development of both software and specific hardware to run the software. Thus, systems engineering encompasses software engineering.

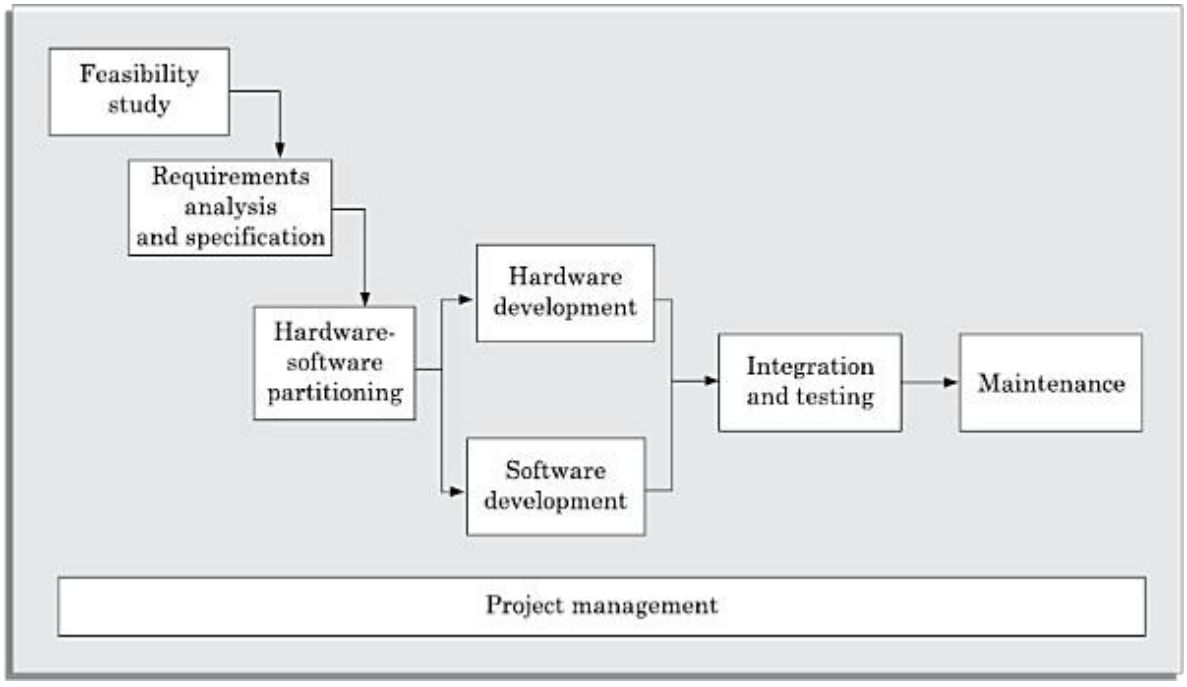
One of the important stages in systems engineering is the stage in which decision is made regarding the parts of the problems that are to be implemented in hardware and the ones that would be implemented in software. This has been represented by the box captioned hardware-software partitioning in below Figure. While partitioning the functions between hardware and software, several trade-offs such as flexibility, cost, speed of operation, etc., need to be considered. The functionality implemented in

hardware run faster. On the other hand, functionalities implemented in software is easier to extend. Further, it is difficult to implement complex functions in hardware. Also, functions implemented in hardware incur extra space, weight, manufacturing cost, and power overhead.

After the hardware-software partitioning stage, development of hardware and software are carried out concurrently. In system engineering, testing the software during development becomes a tricky issue, the hardware on which the software would run and tested would still be under development remember that the hardware and the software are being developed at the same time. To test the software during development, it usually becomes necessary to develop simulators that mimic the features of the hardware being developed.

The software is tested using these simulators. Once both hardware and software development are complete, these are integrated and tested. The project management activity is required throughout the duration of system development as shown in below figure. In this text, we have confined our attention to software engineering only.

Figure: Computer Systems Engineering.



SOFTWARE LIFE CYCLE MODELS

SDLC is a systematic process for building software that ensures the quality and correctness of the software built. SDLC process aims to produce high-quality software that meets customer expectations. The system development should be complete in the pre-defined time frame and cost. SDLC consists of a detailed plan which explains how to plan, build, and maintain specific software. Every phase of the SDLC life Cycle has its own process and deliverables that feed into the next phase. SDLC stands for Software Development Life Cycle and is also referred to as the Application Development life-cycle.

- It offers a basis for project planning, scheduling, and estimating
- Provides a framework for a standard set of activities and deliverables
- It is a mechanism for project tracking and control
- Increases visibility of project planning to all involved stakeholders of the development process
- Increased and enhance development speed
- Improved client relations
- Helps you to decrease project risk and project management plan overhead

SDLC Phases

The entire SDLC process divided into the following Software Development Life Cycle steps:

SDLC Phases

- Phase 1: Requirement collection and analysis
- Phase 2: Feasibility study
- Phase 3: Design
- Phase 4: Coding
- Phase 5: Testing
- Phase 6: Installation/Deployment
- Phase 7: Maintenance

Phase 1: Requirement collection and analysis

The requirement is the first stage in the SDLC process. It is conducted by the senior team members with inputs from all the stakeholders and domain experts in the industry. Planning for the quality assurance requirements and recognization of the risks involved is also done at this stage.

This stage gives a clearer picture of the scope of the entire project and the anticipated issues, opportunities, and directives which triggered the project. Requirements Gathering stage need teams to get detailed and precise requirements. This helps companies to finalize the necessary timeline to finish the work of that system.

Phase 2: Feasibility study

Once the requirement analysis phase is completed the next sdlc step is to define and document software needs. This process conducted with the help of 'Software Requirement Specification'

document also known as 'SRS' document. It includes everything which should be designed and developed during the project life cycle.

There are mainly five types of feasibilities checks:

1. **Economic:** Can we complete the project within the budget or not?
2. **Legal:** Can we handle this project as cyber law and other regulatory framework/compliances?
3. **Operation feasibility:** Can we create operations which are expected by the client?
4. **Technical:** Need to check whether the current computer system can support the software
5. **Schedule:** Decide that the project can be completed within the given schedule or not.

Phase 3: Design

In this third phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture. This design phase serves as input for the next phase of the model. There are two kinds of design documents developed in this phase:

High-Level Design (HLD)

- Brief description and name of each module
- An outline about the functionality of every module
- Interface relationship and dependencies between modules
- Database tables identified along with their key elements
- Complete architecture diagrams along with technology details

Low-Level Design (LLD)

- Functional logic of the modules
- Database tables, which include type and size
- Complete detail of the interface
- Addresses all types of dependency issues
- Listing of error messages
- Complete input and outputs for every module

Phase 4: Coding

Once the system design phase is over, the next phase is coding. In this phase, developers start build the entire system by writing code using the chosen programming language. In the coding phase, tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the Software Development Life Cycle process. In this phase, Developer needs to follow certain predefined coding guidelines. They also need to use programming tools like compiler, interpreters, debugger to generate and implement the code.

Phase 5: Testing

Once the software is complete, and it is deployed in the testing environment. The testing team starts testing the functionality of the entire system. This is done to verify that the entire application works according to the customer requirement. During this phase, QA and testing team may find some bugs/defects which they communicate to developers. The development team fixes the bug and send back

to QA for a re-test. This process continues until the software is bug-free, stable, and working according to the business needs of that system.

Phase 6: Installation/Deployment

Once the software testing phase is over and no bugs or errors left in the system then the final deployment process starts. Based on the feedback given by the project manager, the final software is released and checked for deployment issues if any.

Phase 7: Maintenance

Once the system is deployed, and customers start using the developed system, following 3 activities occur

- Bug fixing – bugs are reported because of some scenarios which are not tested at all
- Upgrade – Upgrading the application to the newer versions of the Software
- Enhancement – Adding some new features into the existing software The main focus of this SDLC phase is to ensure that needs continue to be met and that the system continues to perform as per the specification mentioned in the first phase.

Software modeling is the process of creating abstract representations of a software system. These models serve as blueprints that guide developers, designers, and stakeholders through the system's structure, behavior, and functionality. By using diagrams and various modeling languages, software modeling helps in visualizing and understanding the complex aspects of the software, making it easier to plan, develop, and manage the system. Choosing the right model is very important for the development of a software product or application. Development and testing processes are carried out based on the model.

PROCESS VERSUS METHODOLOGY

- Process: A broader framework encompassing all activities from inception to retirement. It guides the overall development approach.
- Methodology: A specific set of steps for a single or a few activities within the development process.

Importance of Process:

- Systematic and Disciplined Development: Processes provide a structured approach, reducing chaos and improving efficiency.
- Essential for Team Projects: In team settings, a process ensures coordination, communication, and integration of individual contributions.
- Reduces Risks: A well-defined process helps identify and mitigate potential problems early on.

Challenges of Ad-Hoc Development:

- Lack of Coordination: Without a shared process, team members may make conflicting assumptions, leading to integration issues.

- Unpredictable Outcomes: Uncontrolled development can lead to delays, cost overruns, and ultimately, project failure.

Programming-in-the-Small vs. Programming-in-the-Large:

- Small-Scale: Individual projects may be successful with less formal processes.
- Large-Scale: Team projects require a well-defined SDLC for success.

In essence, the text emphasizes that while individual programmers may succeed with less structured approaches, a robust software development process is crucial for the successful completion of complex projects involving multiple developers.

WATERFALL MODEL AND ITS EXTENSIONS

The waterfall model and its derivatives were extremely popular in the 1970s and still are heavily being used across many development projects. The waterfall model is possibly the most obvious and intuitive way in which software can be developed through team effort. We can think of the waterfall model as a generic model that has been extended in many ways for catering to certain specific software development situations to realise all other software life cycle models. For this reason, after discussing the classical and iterative waterfall models, we discuss its various extensions.

CLASSICAL WATERFALL MODEL

Classical waterfall model is intuitively the most obvious way to develop software. It is simple but idealistic. In fact, it is hard to put this model into use in any non-trivial software development project. One might wonder if this model is hard to use in practical development projects, then why study it at all? The reason is that all other life cycle models can be thought of as being extensions of the classical water fall model.

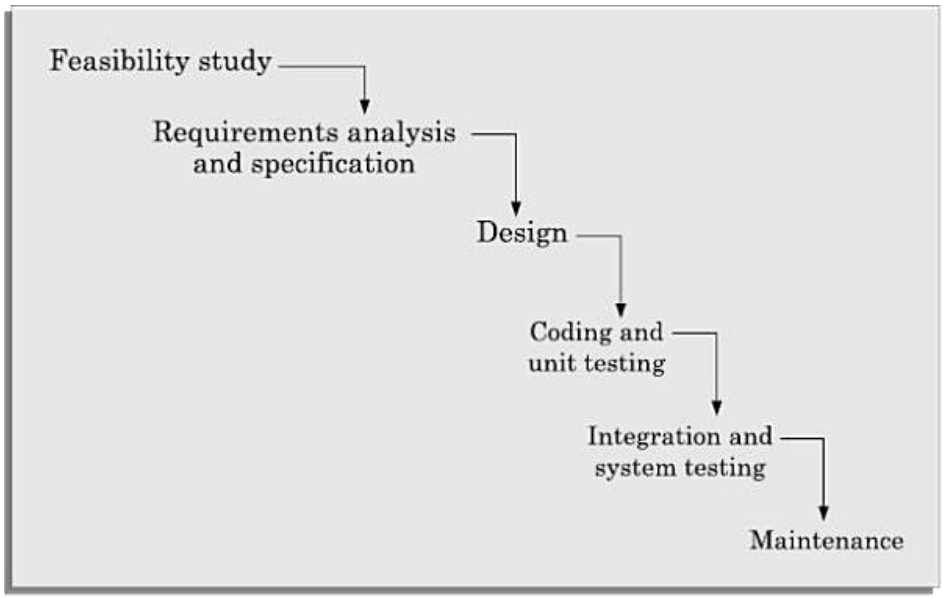
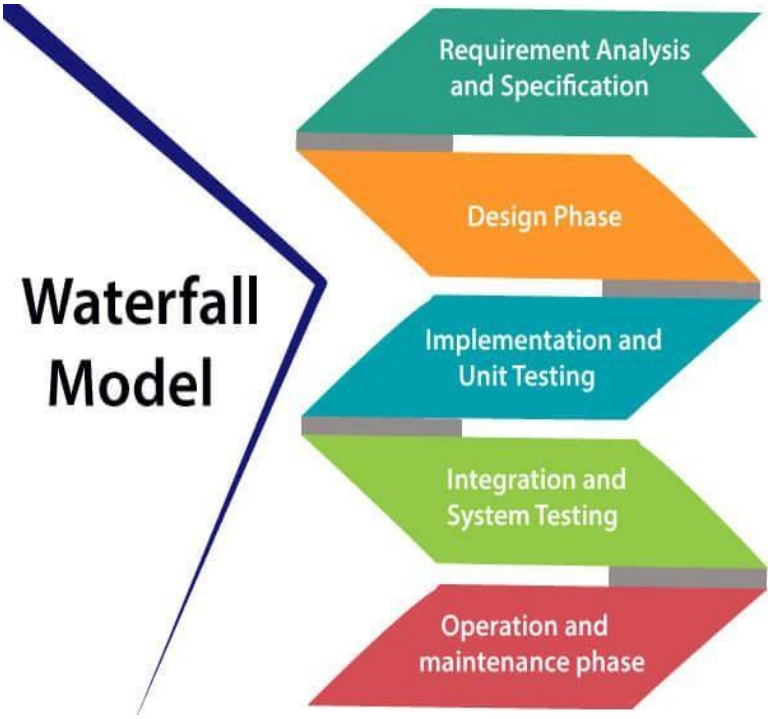


Figure2.1: Classical waterfall model.

Therefore, it makes sense to first understand the classical waterfall model, in order to be able to develop a proper understanding of other life cycle models.

The classical waterfall model divides the life cycle into a set of phases as shown in Figure 2.1.It can be easily observed from this figure that the diagrammatic representation of the classical waterfall model resembles a multi-level waterfall. This resemblance justifies the name of the model.



This model has five phases: Requirements analysis and specification, design, implementation, and unit testing, integration and system testing, and operation and maintenance. The steps always follow in this order and do not overlap. The developer must complete every phase before the next phase begins. This model is named "**Waterfall Model**", because its diagrammatic representation resembles a cascade of waterfalls.

1. Requirements analysis and specification phase:

The aim of this phase is to understand the exact requirements of the customer and to document them properly. Both the customer and the software developer work together so as to document all the functions, performance, and interfacing requirement of the software. It describes the "what" of the system to be produced and not "how."In this phase, a large document called **Software Requirement Specification (SRS)** document is created which contained a detailed description of what the system will do in the common language.

2. Design Phase:

This phase aims to transform the requirements gathered in the SRS into a suitable form which permits further coding in a programming language. It defines the overall software architecture together with high level and detailed design. All this work is documented as a Software Design Document (SDD).

3. Implementation and unit testing: During this phase, design is implemented. If the SDD is complete, the implementation or coding phase proceeds smoothly, because all the information needed by software developers is contained in the SDD.

During testing, the code is thoroughly examined and modified. Small modules are tested in isolation initially. After that these modules are tested by writing some overhead code to check the interaction between these modules and the flow of intermediate output.

4. Integration and System Testing: This phase is highly crucial as the quality of the end product is determined by the effectiveness of the testing carried out. The better output will lead to satisfied customers, lower maintenance costs, and accurate results. Unit testing determines the efficiency of individual modules. However, in this phase, the modules are tested for their interactions with each other and with the system.

5. Operation and maintenance phase: Maintenance is the task performed by every user once the software has been delivered to the customer, installed, and operational.

When to use SDLC Waterfall Model?

Some Circumstances where the use of the Waterfall model is most suited are:

- When the requirements are constant and not changed regularly.
- A project is short
- The situation is calm
- Where the tools and technology used is consistent and is not changing
- When resources are well prepared and are available to use.

Advantages of Waterfall model

- This model is simple to implement also the number of resources that are required for it is minimal.
- The requirements are simple and explicitly declared; they remain unchanged during the entire project development.
- The start and end points for each phase is fixed, which makes it easy to cover progress.

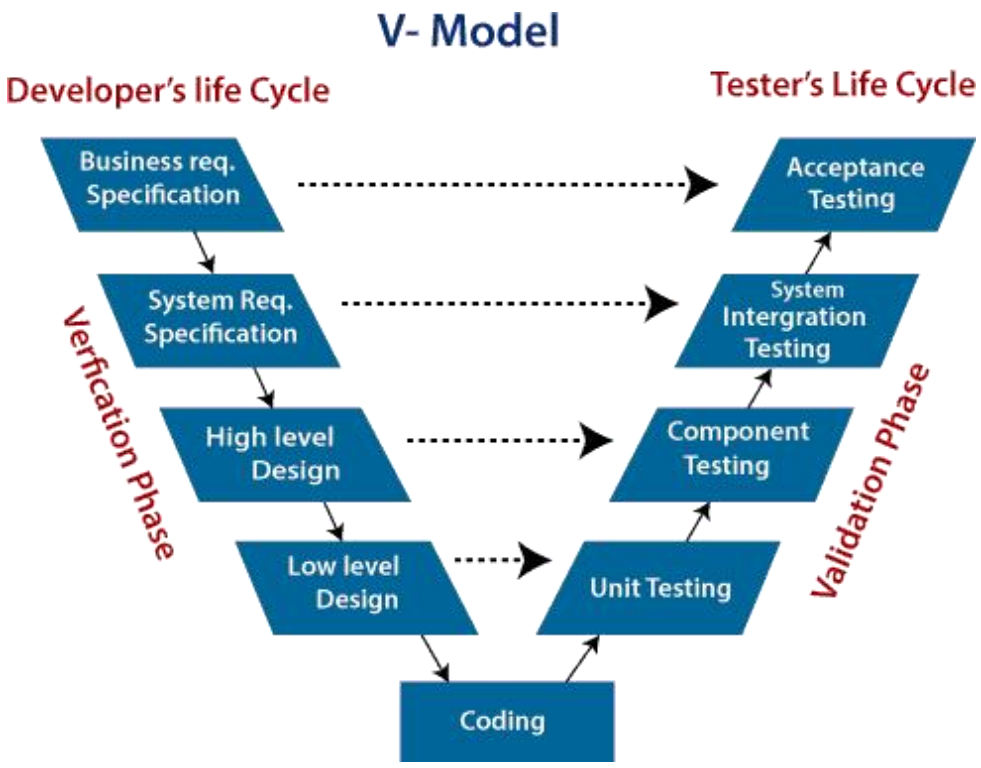
- The release date for the complete product, as well as its final cost, can be determined before development.
- It gives easy to control and clarity for the customer due to a strict reporting system.

Disadvantages of Waterfall model

- In this model, the risk factor is higher, so this model is not suitable for more significant and complex projects.
- This model cannot accept the changes in requirements during development.
- It becomes tough to go back to the phase. For example, if the application has now shifted to the coding phase, and there is a change in requirement, It becomes tough to go back and change it.
- Since the testing done at a later stage, it does not allow identifying the challenges and risks in the earlier phase, so the risk reduction strategy is difficult to prepare.

V-MODEL

V-Model also referred to as the Verification and Validation Model. In this, each phase of SDLC must complete before the next phase starts. It follows a sequential design process same as the waterfall model. Testing of the device is planned in parallel with a corresponding stage of development.



Verification: It involves a static analysis method (review) done without executing code. It is the process of evaluation of the product development process to find whether specified requirements meet.

Validation: It involves dynamic analysis method (functional, non-functional), testing is done by executing code. Validation is the process to classify the software after the completion of the development process to determine whether the software meets the customer expectations and requirements.

So V-Model contains Verification phases on one side of the Validation phases on the other side. Verification and Validation process is joined by coding phase in V-shape. Thus it is known as V-Model.

There are the various phases of Verification Phase of V-model:

1. **Business requirement analysis:** This is the first step where product requirements understood from the customer's side. This phase contains detailed communication to understand customer's expectations and exact requirements.
2. **System Design:** In this stage system engineers analyze and interpret the business of the proposed system by studying the user requirements document.
3. **Architecture Design:** The baseline in selecting the architecture is that it should understand all which typically consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc. The integration testing model is carried out in a particular phase.
4. **Module Design:** In the module design phase, the system breaks down into small modules. The detailed design of the modules is specified, which is known as Low-Level Design
5. **Coding Phase:** After designing, the coding phase is started. Based on the requirements, a suitable programming language is decided. There are some guidelines and standards for coding. Before checking in the repository, the final build is optimized for better performance, and the code goes through many code reviews to check the performance.

There are the various phases of Validation Phase of V-model:

1. **Unit Testing:** In the V-Model, Unit Test Plans (UTPs) are developed during the module design phase. These UTPs are executed to eliminate errors at code level or unit level. A unit is the smallest entity which can independently exist, e.g., a program module. Unit testing verifies that the smallest entity can function correctly when isolated from the rest of the codes/ units.
2. **Integration Testing:** Integration Test Plans are developed during the Architectural Design Phase. These tests verify that groups created and tested independently can coexist and communicate among themselves.
3. **System Testing:** System Tests Plans are developed during System Design Phase. Unlike Unit and Integration Test Plans, System Tests Plans are composed by the client?s business team. System Test ensures that expectations from an application developer are met.

4. **Acceptance Testing:** Acceptance testing is related to the business requirement analysis part. It includes testing the software product in user atmosphere. Acceptance tests reveal the compatibility problems with the different systems, which is available within the user atmosphere. It conjointly discovers the non-functional problems like load and performance defects within the real user atmosphere.

When to use V-Model?

- When the requirement is well defined and not ambiguous.
- The V-shaped model should be used for small to medium-sized projects where requirements are clearly defined and fixed.
- The V-shaped model should be chosen when sample technical resources are available with essential technical expertise.

Advantage (Pros) of V-Model:

1. Easy to Understand.
2. Testing Methods like planning, test designing happens well before coding.
3. This saves a lot of time. Hence a higher chance of success over the waterfall model.
4. Avoids the downward flow of the defects.
5. Works well for small plans where requirements are easily understood.

Disadvantage (Cons) of V-Model:

1. Very rigid and least flexible.
2. Not a good for a complex project.
3. Software is developed during the implementation stage, so no early prototypes of the software are produced.
4. If any changes happen in the midway, then the test documents along with the required documents, has to be updated.

INCREMENTAL MODEL:

Incremental Model is a process of software development where requirements divided into multiple standalone modules of the software development cycle. In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release. The process continues until the complete system achieved.

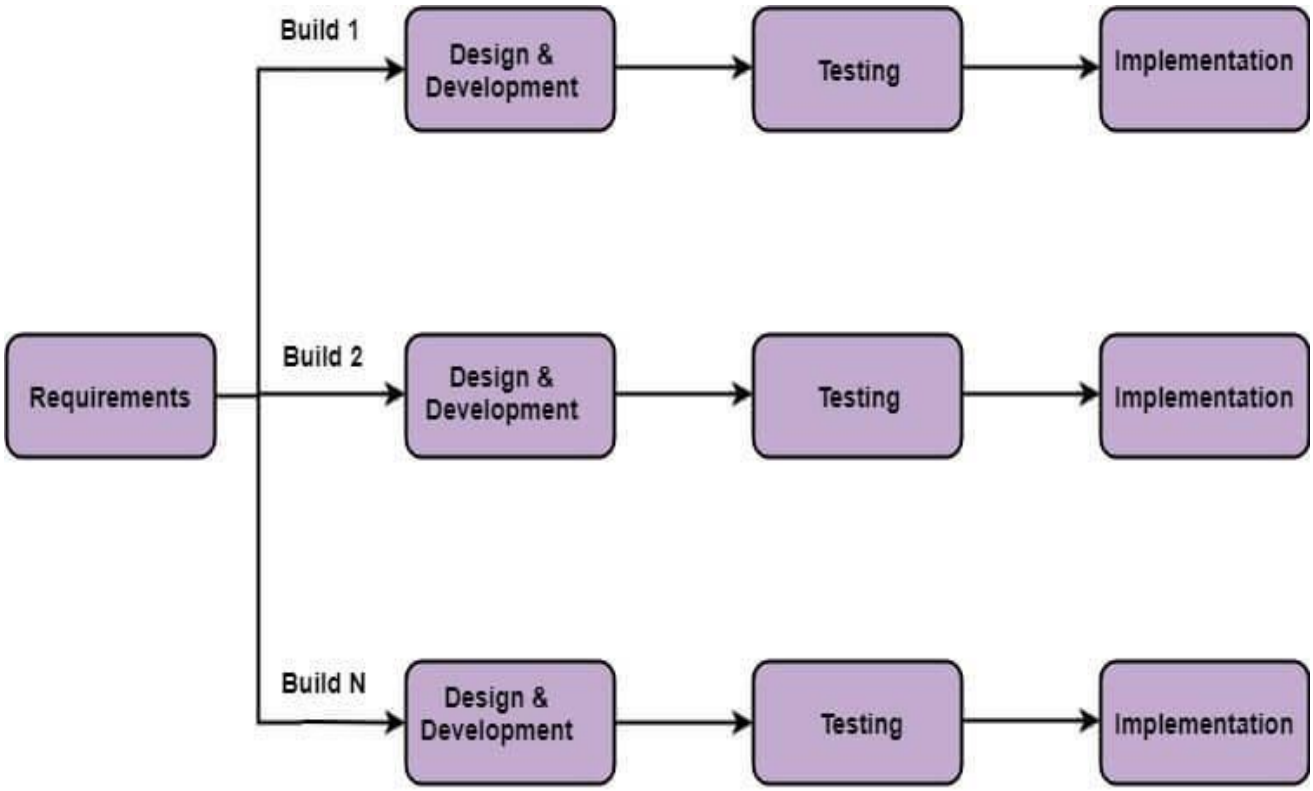


Fig: Incremental Model

The various phases of incremental model are as follows:

1. Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase performs a crucial role.

2. Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

3. Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

4. Implementation: Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

When we use the Incremental Model?

- When the requirements are superior.
- A project has a lengthy development schedule.
- When Software team are not very well skilled or trained.
- When the customer demands a quick release of the product.
- You can develop prioritized requirements first.

Advantage of Incremental Model

- Errors are easy to be recognized.
- Easier to test and debug
- More flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.

Disadvantage of Incremental Model

- Need for good planning
- Total Cost is high.
- Well defined module interfaces are needed.

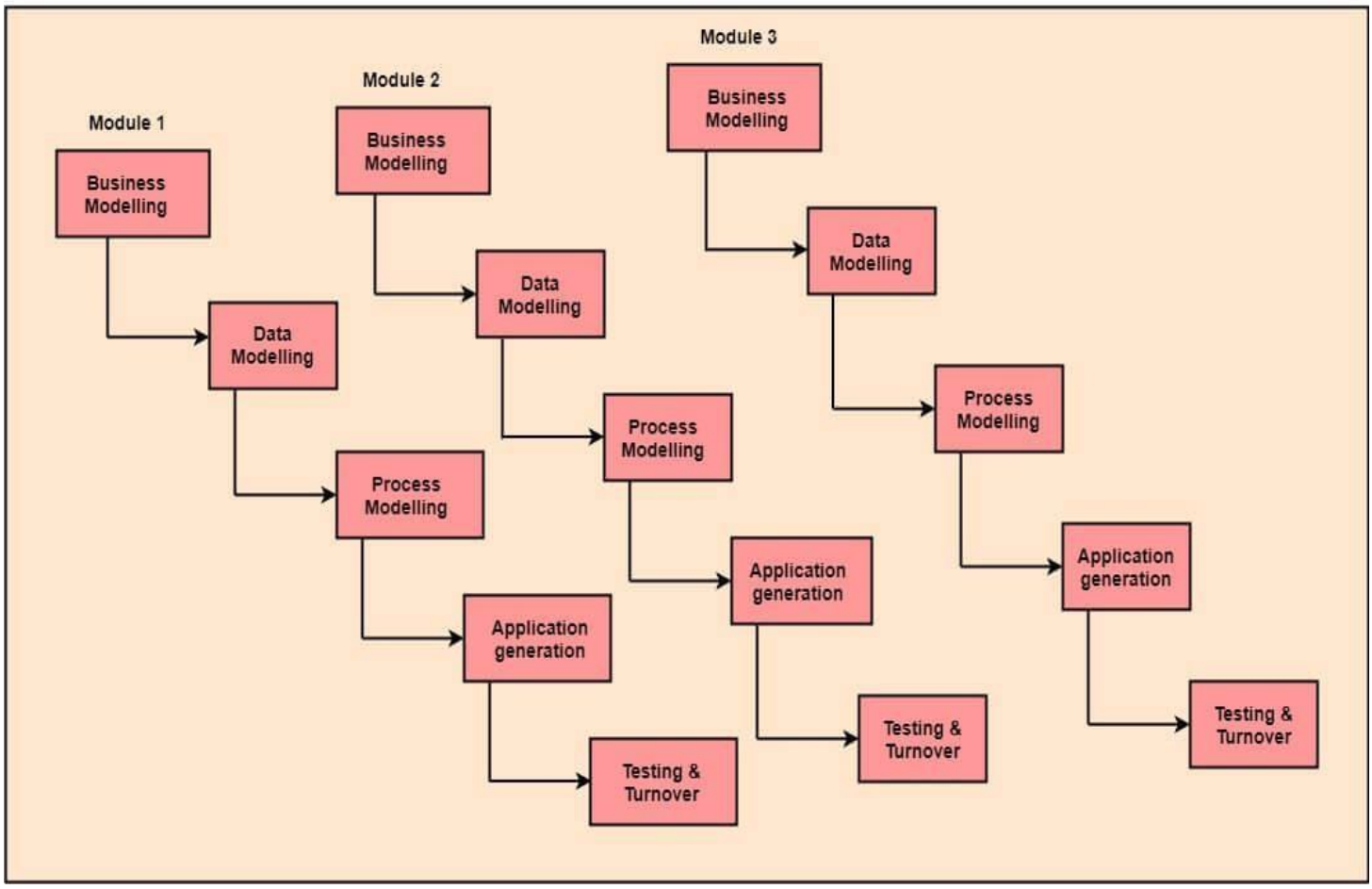
RAD (RAPID APPLICATION DEVELOPMENT) MODEL:

RAD is a linear sequential software development process model that emphasizes a concise development cycle using an element based construction approach. If the requirements are well understood and described, and the project scope is a constraint, the RAD process enables a development team to create a fully functional system within a concise time period.

RAD (Rapid Application Development) is a concept that products can be developed faster and of higher quality through:

- Gathering requirements using workshops or focus groups
- Prototyping and early, reiterative user testing of designs
- The re-use of software components
- A rigidly paced schedule that refers design improvements to the next product version
- Less formality in reviews and other team communication

Fig: RAD Model



The various phases of RAD are as follows:

1. **Business Modelling:** The information flow among business functions is defined by answering questions like what data drives the business process, what data is generated, who generates it, where does the information go, who process it and so on.

2. **Data Modelling:** The data collected from business modeling is refined into a set of data objects (entities) that are needed to support the business. The attributes (character of each entity) are identified, and the relation between these data objects (entities) is defined.

3. **Process Modelling:** The information object defined in the data modeling phase are transformed to achieve the data flow necessary to implement a business function. Processing descriptions are created for adding, modifying, deleting, or retrieving a data object.

4. **Application Generation:** Automated tools are used to facilitate construction of the software; even they use the 4th GL techniques.

5. Testing & Turnover: Many of the programming components have already been tested since RAD emphasis reuse. This reduces the overall testing time. But the new part must be tested, and all interfaces must be fully exercised.

When to use RAD Model?

- When the system should need to create the project that modularizes in a short span time (2-3 months).
- When the requirements are well-known.
- When the technical risk is limited.
- When there's a necessity to make a system, which modularized in 2-3 months of period.
- It should be used only if the budget allows the use of automatic code generating tools.

Advantage of RAD Model

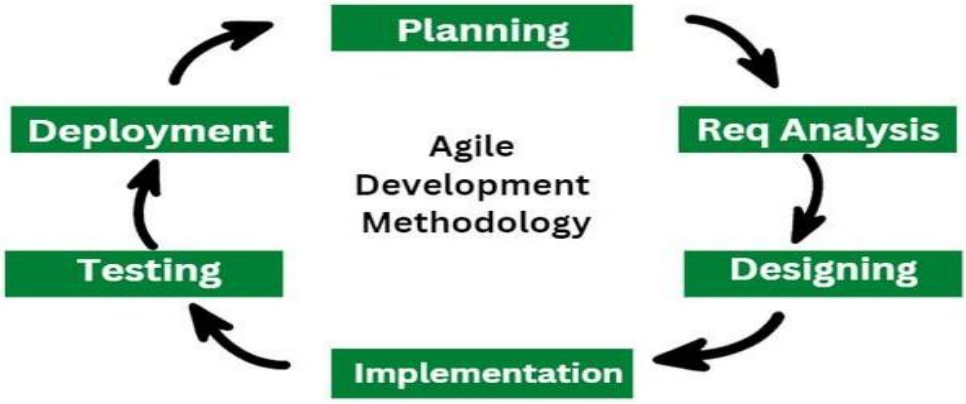
- This model is flexible for change.
- In this model, changes are adoptable.
- Each phase in RAD brings highest priority functionality to the customer.
- It reduced development time.
- It increases the reusability of features.

Disadvantage of RAD Model

- It required highly skilled designers.
- All application is not compatible with RAD.
- For smaller projects, we cannot use the RAD model.
- On the high technical risk, it's not suitable.
- Required user involvement.

AGILE MODEL:

The meaning of Agile is swift or versatile."**Agile process model**" refers to a software development approach based on iterative development. Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning. The project scope and requirements are laid down at the beginning of the development process. Plans regarding the number of iterations, the duration and the scope of each iteration are clearly defined in advance.



Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks. The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements. Each iteration involves a team working through a full software development life cycle including planning, requirements analysis, design, coding, and testing before a working product is demonstrated to the client.



Fig. Agile Model

Phases of Agile Model:

Following are the phases in the Agile model are as follows:

1. Requirements gathering
2. Design the requirements
3. Construction/ iteration
4. Testing/ Quality assurance
5. Deployment
6. Feedback

1. Requirements gathering: In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility.

2. Design the requirements: When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.

3. Construction/ iteration: When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.

4. Testing: In this phase, the Quality Assurance team examines the product's performance and looks for the bug.

5. Deployment: In this phase, the team issues a product for the user's work environment.

6. Feedback: After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Agile Testing Methods:

- Scrum
- Crystal
- Dynamic Software Development Method(DSDM)
- Feature Driven Development(FDD)
- Lean Software Development
- eXtreme Programming(XP)

Scrum

SCRUM is an agile development process focused primarily on ways to manage tasks in team-based development conditions.

There are three roles in it, and their responsibilities are:

- **Scrum Master:** The scrum can set up the master team, arrange the meeting and remove obstacles for the process
- **Product owner:** The product owner makes the product backlog, prioritizes the delay and is responsible for the distribution of functionality on each repetition.
- **Scrum Team:** The team manages its work and organizes the work to complete the sprint or cycle.

eXtreme Programming(XP)

This type of methodology is used when customers are constantly changing demands or requirements, or when they are not sure about the system's performance.

Crystal:

There are three concepts of this method-

1. Chartering: Multi activities are involved in this phase such as making a development team, performing feasibility analysis, developing plans, etc.
2. Cyclic delivery: under this, two more cycles consist, these are:
    - Team updates the release plan.
    - Integrated product delivers to the users.
3. Wrap up: According to the user environment, this phase performs deployment, post-deployment.

Dynamic Software Development Method(DSDM):

DSDM is a rapid application development strategy for software development and gives an agile project distribution structure. The essential features of DSDM are that users must be actively connected, and teams have been given the right to make decisions. The techniques used in DSDM are:

1. Time Boxing
2. MoSCoW Rules
3. Prototyping

The DSDM project contains seven stages:

1. Pre-project
2. Feasibility Study
3. Business Study

4. Functional Model Iteration
5. Design and build Iteration
6. Implementation
7. Post-project

Feature Driven Development (FDD):
This method focuses on "Designing and Building" features. In contrast to other smart methods, FDD describes the small steps of the work that should be obtained separately per function.

Lean Software Development:
Lean software development methodology follows the principle "just in time production." The lean method indicates the increasing speed of software development and reducing costs. Lean development can be summarized in seven phases.

1. Eliminating Waste
2. Amplifying learning
3. Defer commitment (deciding as late as possible)
4. Early delivery
5. Empowering the team
6. Building Integrity
7. Optimize the whole

When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

Advantages of Agile Method:
1. Frequent Delivery
2. Face-to-Face Communication with clients.
3. Efficient design and fulfils the business requirement.
4. Anytime changes are acceptable.
5. It reduces total development time.

Disadvantages of Agile Model:
1. Due to the shortage of formal documents, it creates confusion and crucial decisions taken throughout various phases can be misinterpreted at any time by different team members.
2. Due to the lack of proper documentation, once the project completes and the developers allotted to another project, maintenance of the finished project can become a difficulty.

SPIRAL MODEL:

The spiral model, initially proposed by Boehm, is an evolutionary software process model that couples the iterative feature of prototyping with the controlled and systematic aspects of the linear sequential model. It implements the potential for rapid development of new versions of the software. Using the spiral model, the software is developed in a series of incremental releases. During the early iterations, the additional release may be a paper model or prototype. During later iterations, more and more complete versions of the engineered system are produced.

The Spiral Model is shown in fig:

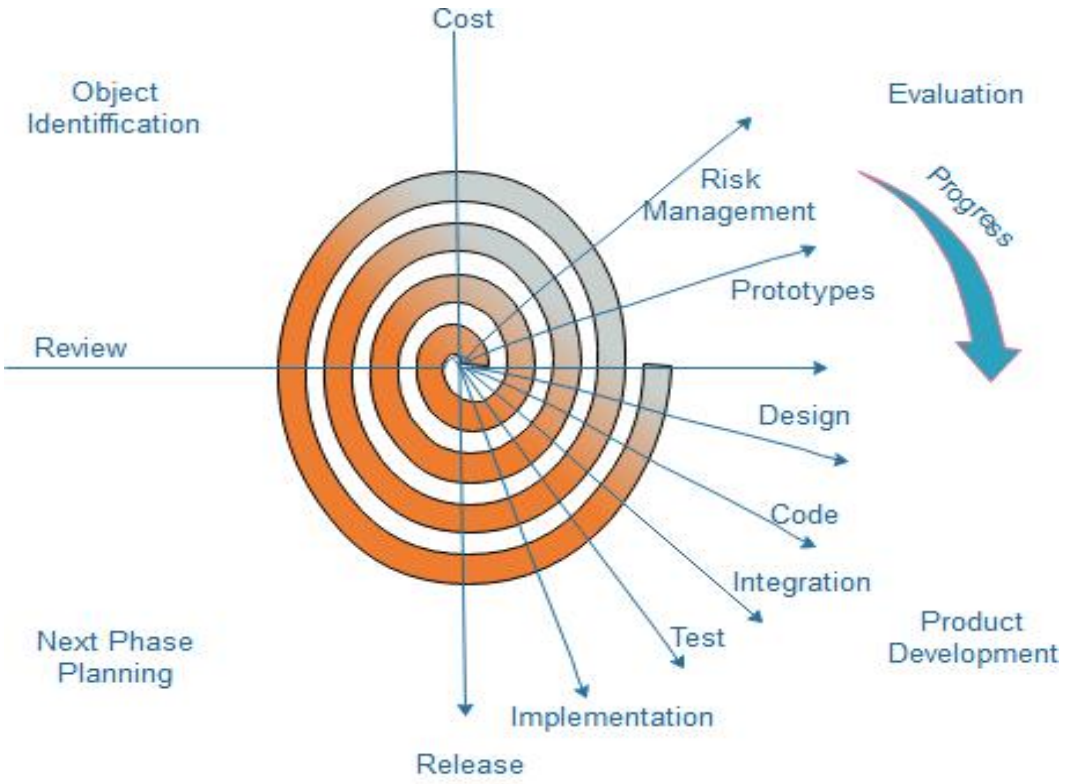
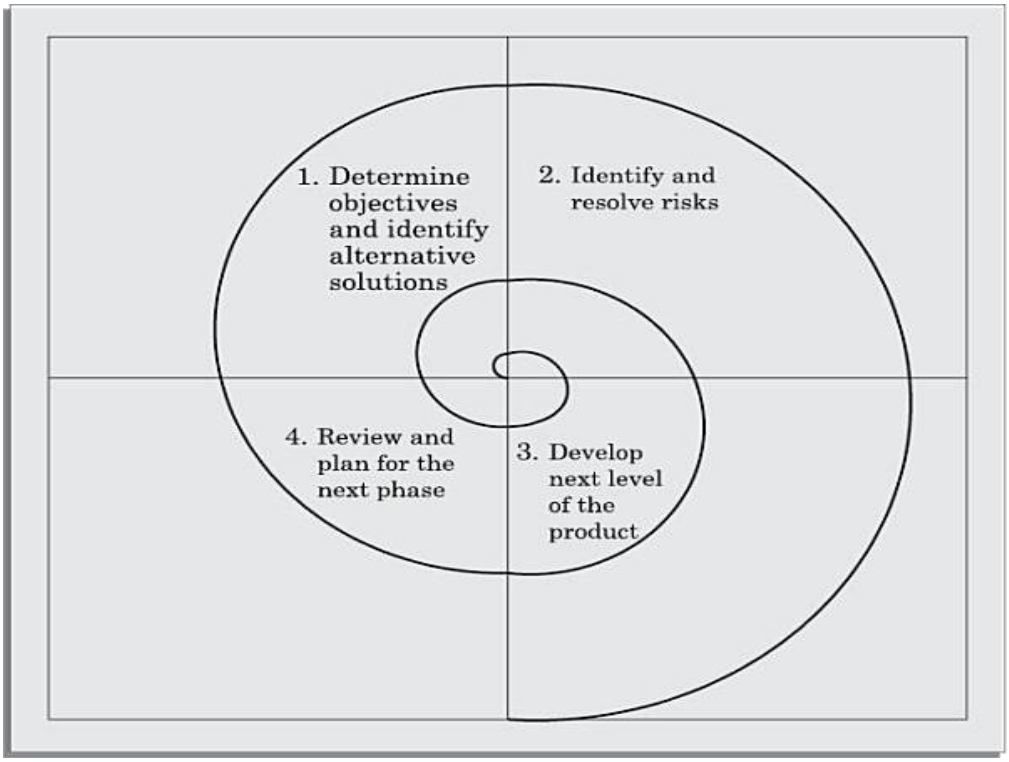


Fig. Spiral Model



Each cycle in the spiral is divided into four parts:

Objective setting: Each cycle in the spiral starts with the identification of purpose for that cycle, the various alternatives that are possible for achieving the targets, and the constraints that exists.

Risk Assessment and reduction: The next phase in the cycle is to calculate these various alternatives based on the goals and constraints. The focus of evaluation in this stage is located on the risk perception for the project.

Development and validation: The next phase is to develop strategies that resolve uncertainties and risks. This process may include activities such as benchmarking, simulation, and prototyping.

Planning: Finally, the next step is planned. The project is reviewed, and a choice made whether to continue with a further period of the spiral. If it is determined to keep, plans are drawn up for the next step of the project.

The development phase depends on the remaining risks. For example, if performance or user-interface risks are treated more essential than the program development risks, the next phase may be an evolutionary development that includes developing a more detailed prototype for solving the risks.

The **risk-driven** feature of the spiral model allows it to accommodate any mixture of a specification-oriented, prototype-oriented, simulation-oriented, or another type of approach. An essential element of the model is that each period of the spiral is completed by a review that includes all the products

developed during that cycle, including plans for the next cycle. The spiral model works for development as well as enhancement projects.

When to use Spiral Model?

- When deliverance is required to be frequent.
- When the project is large
- When requirements are unclear and complex
- When changes may require at any time
- Large and high budget projects

Advantages

- High amount of risk analysis
- Useful for large and mission-critical projects.

Disadvantages

- Can be a costly model to use.
- Risk analysis needed highly particular expertise
- Doesn't work well for smaller projects.

EXPLORATORY MODEL:

Exploratory program development style refers to an informal development style **or** builds and fix the style in which the programmer uses his own intuition to develop a program rather than making use of the systematic body of knowledge which is categorized under the software engineering discipline. This style of development gives complete freedom to programmers to choose activities which they like to develop software. This dirty program is quickly developed and bugs are fixed whenever it arises.
This style does not offer any rules to start developing any software. The following block diagram will clear some facts relating to this model :

In the above diagram, the first block is the **initial briefing by the customer** i.e brief introduction of the problem by the customer. After the briefing, control goes to **initial coding** i.e. as soon as the developer or programmer knew about the problem he starts coding to develop a working program without considering any kind of requirement analysis. After this, the program will be **tested** i.e bugs found and they are getting fixed by programmers. This cycle continues until satisfactory code is not obtained. After finding satisfactory code, development gets completed.

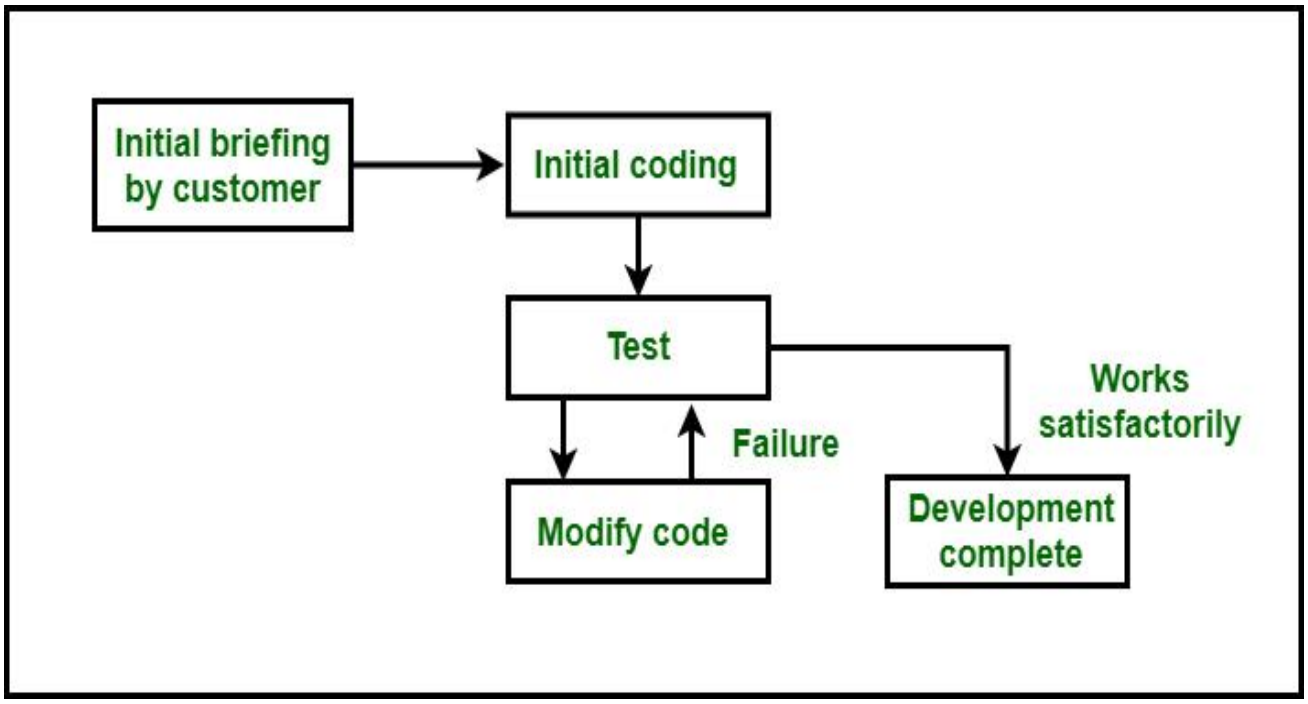


Figure – Exploratory style of software development

Usage:

This style of software development is only used for the development of small programs. Nowadays, this style is only used by students in their labs to complete their assignments only. This style is not really used in industries nowadays.